ANSUT
AGENCE NATIONALE DU SERVICE UNIVERSEL DES TÉLÉCOMMUNICATIONS-TIC

A R C H I T E C T U R E D O C U M E N T

Architecture de la plateforme

Architecture technique et fonctionnelle — Mon Toit CI

Mon Toit CI · Plateforme de location immobilière — Côte d'Ivoire
Branche 4.0.0 · Next.js 16 · Supabase · Capacitor 8
Version 1.0 — 3 novembre 2026

Table des matières

1. Présentation générale 1

1.1 Contexte et objectifs 1

1.2 Chiffres clés de la plateforme 2

1.3 Principes directeurs 2

2. Architecture fonctionnelle 3

2.1 Les six acteurs 3

2.2 Contrôle d'accès et matrice de permissions 3

2.3 Modules fonctionnels par acteur 4

2.4 Fonctionnalités transverses implémentées 4

2.5 Flux utilisateurs clés 4

2.5.1 Parcours du locataire 5

2.5.2 Parcours du bailleur (propriétaire) 6

2.5.3 Parcours de l'agence 7

2.5.4 Parcours de l'agent de terrain 7

2.5.5 Parcours de l'administrateur 7

3. Architecture technique 8

3.1 Vue d'ensemble en couches 8

3.2 Stack technique 8

3.3 Frontend — application mono-page 8

3.4 Couche API Next.js 9

3.5 Middleware et contrôle d'accès 9

3.6 Modèle de données 9

3.7 Authentification et sessions 10

3.8 Edge Functions Supabase 10

3.9 Stockage (Supabase Storage) 10

3.10 Sécurité....11
4. Intégrations externes....12
4.1 CRYPTONEO / ANSUT — signature électronique....12
4.2 Intouch — paiements mobile money....12
4.3 Vérification d'identité (KYC)....13
4.4 Communications (e-mail, SMS, WhatsApp)....13
4.5 Intelligence artificielle....13
4.6 Cartographie et géolocalisation....13
5. Application mobile — Capacitor 8....14
5.1 Configuration....14
5.2 Plugins et habillage natif....14
5.3 Stratégie de build (ADR-0004)....14
6. Qualité, CI/CD et exploitation....14
6.1 Chaîne d'intégration continue....15
6.2 Conventions de développement....15
6.3 Stratégie de tests....15
6.4 Déploiement et exploitation....16
6.5 Tâches planifiées (pg_cron)....16
6.6 Pilotage projet (.ai/)....16
7. Environnement et configuration....17
8. Décisions d'architecture (ADRs)....17
9. Dette technique, points d'attention et évolutions....17
9.1 Points d'attention identifiés....18
9.2 Restes à faire (TEAM_STATUS)....18
9.3 Axes d'évolution recommandés....18
10. Annexes....19
10.1 Documents de référence internes....19
10.2 Périmètre des migrations récentes (waves 2-3)....21

Remarque : cette table des matières est générée par champs. Après toute modification du document, faites un clic droit sur la table puis « Mettre à jour les champs » pour actualiser la pagination.

1. Présentation générale

1.1 Contexte et objectifs

Mon Toit CI est une plateforme web et mobile de mise en relation pour la location immobilière en Côte d'Ivoire. Elle couvre l'ensemble du cycle locatif : publication et vérification des biens, constitution et validation des dossiers de location, organisation des visites, signature électronique des baux et des mandats de gestion, encaissement des loyers par mobile money, émission des quittances, gestion des états des lieux, de la maintenance, des litiges et de la restitution du dépôt de garantie.

La plateforme s'appuie sur un tiers de confiance (TIERS_CONFIANCE, « TC ») qui valide les dossiers des locataires, les biens des propriétaires, les documents de propriété et les identités, et s'appuie sur des agents de terrain sans compte pour les vérifications physiques et les états des lieux. Un espace d'administration assure la supervision, la modération, la sécurité et la configuration.

Le présent document décrit l'architecture fonctionnelle (acteurs, permissions, modules, flux) et l'architecture technique (couches, données, API, sécurité, intégrations, exploitation) de la branche 4.0.0 telle qu'elle est réellement implémentée dans le dépôt. Il s'appuie sur les fichiers de pilotage du dossier .ai/ (ACTORS.md, ARCHITECTURE.md, ADRs, SPECS, TEAM_STATUS.md) et sur l'inventaire du code source.

1.2 Chiffres clés de la plateforme

Tableau 1 — Chiffres clés de l'implémentation (branche 4.0.0)

Indicateur	Valeur	Détail
Acteurs	6	5 rôles en base (enum user_role) + visiteur implicite non authentifié
Routes API Next.js	167	Fichiers route.ts sous src/app/api (arbre versionné)
Edge Functions Supabase	60	Runtime Deno, hors module partagé _shared (16 modules)
Migrations SQL	102	supabase/migrations, de 2026011701 à 20261027000060
Tables actives	≈ 51	7 tables V1 legacy exclues du comptage

Indicateur	Valeur	Détail
Enums Postgres	36	Migration 20260518131900 + rental_file_type
Politiques RLS	247+	Deuxième ligne de défense (cf. section 3.10)
Buckets Storage	11	8 privés (URLs signées) + 3 publics catalogue
Tâches planifiées	8	pg_cron, protégées par CRON_SECRET
Composants dashboard	≈ 110	5 dossiers par acteur sous src/components/dashboard/
Tâches spec implémentées	3 / 3	FUN-001 quittances, FUN-002 dossiers multiples, TC-001 agents terrain

1.3 Principes directeurs

- Application mono-page (SPA) : une seule page App Router applicative (src/app/page.tsx) dispatche les vues côté client ; une seule page publique dédiée (/m/[token]) pour les agents de terrain (décision ADR-0001).
- Sécurité en profondeur : contrôle d'accès applicatif systématique dans les routes API (pattern resolveRequestUser + scoping par requête), politiques RLS comme deuxième ligne de défense, secrets hachés avec pepper (OTP, tokens de mission), webhook de paiement fail-closed vérifié par HMAC.
- Contrat partagé entre runtimes : toute logique pure (constantes, types, formes d'API) réside dans supabase/functions/_shared/ et est importée à la fois par le runtime Node (Next.js) et le runtime Deno (Edge Functions) — règle DAT-004, appliquée à l'intégration CRYPTONEO.
- Services par acteur : hooks et services spécialisés par acteur (tenant, proprietaire, agency, tc, admin), migration progressive vers React Query (ADR-0003).
- Données et fichiers chez Supabase : Postgres, Auth, Storage, Realtime et Edge Functions forment le socle managé ; l'application Next.js n'héberge aucun état persistant.

2. Architecture fonctionnelle

2.1 Les six acteurs

Les rôles applicatifs sont portés par l'enum Postgres user_role : LOCATAIRE, PROPRIETAIRE, AGENCE, ADMIN et TIERS_CONFIANCE. La table users porte deux colonnes : role (rôle principal) et active_role (rôle actif dans la session). Le rôle VISITEUR est implicite : il correspond à l'utilisateur non authentifié, autorisé en lecture publique sur le catalogue de biens et les avis ; il n'est pas stocké en base. Le changement de rôle actif est servi par la route /api/user/switch-role et reste restreint au couple LOCATAIRE ↔ PROPRIETAIRE (composant RoleSwitcher).

Cas particulier : les agents de terrain du tiers de confiance ne disposent d'aucun compte. Ce sont des fiches de la table verification_agents rattachées à un TC, qui accèdent à leurs missions par des liens à jeton éphémère (cf. section 2.5.4) — décision produit TC-001 dite « Option C ».

Tableau 2 — Acteurs de la plateforme et périmètre fonctionnel

Acteur	Rôle enum	Mission	Composants dédiés
Visiteur	— (implicite)	Consulter le catalogue public, les avis, la documentation légale ; s'inscrire.	src/components/home/ (19), src/components/auth/ (5)
Locataire	LOCATAIRE	Rechercher, constituer son dossier, candidater, visiter, signer le bail, payer, suivre ses quittances.	src/components/ dashboard/locataire/ (25)
Propriétaire	PROPRIETAIRE	Publier des biens, sélectionner les locataires, gérer baux, paiements, finances, mandats.	src/components/ dashboard/proprietaire/ (21)
Agence	AGENCE	Gérer un portefeuille de biens sous mandat, les contrats, l'équipe, les commissions.	src/components/ dashboard/agence/ (14)
Tiers de confiance	TIERS_CONFIANCE	Valider dossiers, biens, identités ; piloter missions, états des lieux, litiges, SLA.	src/components/ dashboard/tc/ (28)

Acteur	Rôle enum	Mission	Composants dédiés
Administrateur	ADMIN	Superviser la plateforme, modérer, gérer les comptes TC, la sécurité, la configuration.	src/components/ dashboard/admin/ (15)

2.2 Contrôle d'accès et matrice de permissions

Le middleware racine (middleware.ts) vérifie le JWT Supabase de chaque requête et applique une cartographie de préfixes par rôle (ROLE_ROUTES) : les espaces API /api/dashboard/{acteur} et /api/{acteur}/ ne sont accessibles qu'au rôle actif correspondant. Un ensemble de préfixes publics (auth, catalogue des biens, avis) et d'environ vingt préfixes protégés (session requise) complète la cartographie. Le contrôle s'effectue soit sur le JWT vérifié via supabase.auth.getUser(), soit sur le cookie de session SMS personnalisé montoit-session.

La matrice détaillée des permissions (35 lignes couvrant biens, dossiers, candidatures, visites, baux, paiements, mandats, missions, messagerie, avis, comptes TC, configuration) est tenue à jour dans .ai/ACTORS.md, section 2 ; elle est cohérente avec les politiques RLS et le scoping des handlers API. Deux règles transverses méritent d'être citées : l'escalade de privilèges à l'inscription est bloquée côté serveur (rôle whitelisté + trigger anti-édition des colonnes sensibles, migrations SEC-001/SEC-004), et le webhook de paiement bascule en mode fail-closed en l'absence de secret HMAC.

2.3 Modules fonctionnels par acteur

Chaque acteur dispose d'un tableau de bord dédié, routé côté client par le composant src/components/dashboard/index.tsx sur la base du rôle actif. La navigation latérale (sidebar.tsx) expose les modules ci-dessous, complétés par des écrans de détail hors sidebar.

Tableau 3 — Modules du tableau de bord par acteur

Acteur	Modules de navigation (sidebar)
Locataire	Vue d'ensemble · Recherche de biens · Favoris · Candidatures · Mes visites · Mes baux · États des lieux · Paiements · Dossier de location (3 types) · Messages · Litiges · Notifications · Paramètres ; hors sidebar : avis, maintenance, historique, score de confiance, détail paiement / candidature / visite / bail.

Acteur	Modules de navigation (sidebar)
Propriétaire	Vue d'ensemble · Mes biens · Mes locataires · Demandes de visite · Candidatures · Mes baux · Mandats · Maintenance · Paiements · Finances · Analytique · Messages · Litiges · Notifications · Paramètres ; hors sidebar : dossier propriétaire, documents du bien, renouvellements, avis, détail locataire, sécurité.
Agence	Vue d'ensemble · Portefeuille · Mandats · Candidatures · Visites · Contrats · Équipe · Finances · Analytique · Communication · Litiges · Marketing · Fiches clients · Paramètres.
Tiers de confiance	Vue d'ensemble · Tous les biens · Vérifications de biens · Validations de dossiers · Validations propriétaires · Validations agences · Vérification ONECI · Utilisateurs · Certifications · États des lieux · Agents · Missions · Documentation · Alertes fraude · Messagerie · Suivi SLA · Litiges · Notifications · Paramètres.
Administrateur	Vue d'ensemble · Utilisateurs · Modération · Modération des avis · Agents TC · Signalements · Litiges · Notifications · Système · Sécurité · Rapports · Paramètres · Sauvegardes.

2.4 Fonctionnalités transverses implémentées

Le tableau ci-dessous recense les fonctionnalités métier majeures du socle commun et leur état réel d'implémentation, avec leurs preuves dans le code.

Tableau 4 — Fonctionnalités transverses et preuves d'implémentation

Fonctionnalité	État	Preuves dans le code
Quittances de loyer (FUN-001)	Implémentée	Table rent_receipts (migration 20261027000010), route /api/payments/[id]/quittance (456 lignes), numérotation QT-YYYY-NNNNNN via séquence, enregistrements immuables ; helper number-to-words-fr.ts (44 tests).

Fonctionnalité	État	Preuves dans le code
Dossiers de location multiples (FUN-002)	Implémentée	Enum rental_file_type (STANDARD / FAMILLE / COLOCATION), colonne rental_files.file_type, unicité d'un brouillon par type, store active-dossier-store.ts, UI rental-files-list.tsx.
Agents de terrain (TC-001)	Implémentée	Table mission_access_tokens (hash sha256 + pepper, TTL, révocation), routes access-link / evidence / missions-token, page publique /m/[token], bucket privé mission-evidence.
Paiements mobile money	Implémentée	Enum payment_method (ORANGE_MONEY, MTN_MOMO, MOOV_MONEY, WAVE) via Intouch ; Edge Functions payment-initiate / callback / transfer ; table payment_attempts (idempotence) ; paiements PARTIELS (amount_paid).
Restitution du dépôt de garantie	Implémentée	Table deposit_refunds (migration 20260910000011), route /api/leases/[id]/deposit-refund, composant partagé deposit-refund-card.tsx.
Signature électronique des baux et mandats	Implémentée	Routes /api/signature/* et /api/leases|mandats/[id]/request-sign-otp|sign ; Edge Functions signature-auth, sign-send-otp, sign, sign-verify, signed-file, generate-certificate ; table signature_aliases (cf. section 4.1).
Vérification d'identité (KYC)	Implémentée	Routes /api/kyc/* vers Edge Functions oneci-* (RNPP Connect) et kyc-face-auth / neoface-verify ; tables oneci_verifications et facial_verifications.

Fonctionnalité	État	Preuves dans le code
États des lieux	Implémentée	Tables inventory_reports + inventory_report_items, composants inventory-report-form (TC et partagé), routes /api/properties/[id]/inventory-reports et /api/tc/inventory-reports.
Messagerie temps réel	Implémentée	Tables conversations, messages, message_attachments (bucket privé), hooks use-realtime-messages, autorisation centralisée messaging-authorization.ts.
Litiges, signalements, fraude	Implémentée	Tables disputes, signalements, fraud_alerts et routes admin / TC correspondantes.
Renouvellements de bail	Implémentée	Routes /api/renewals + Edge Function planifiée remind-lease-expiring.
Commissions d'agence	Implémentée	Table commissions ; création automatique à l'activation d'un bail géré par une agence (createCommissionForActivatedLease).
Alertes de recherche	Implémentée	Table search_alerts + routes /api/search-alerts.
Paiements récurrents	Implémentée (table)	Table recurring_payments + service recurringPaymentService.ts.
Sauvegardes d'administration	Implémentée	Table backups, routes /api/admin/backups (y compris restauration et téléchargement).
Assistant conversationnel SUTA	Implémentée	Route /api/suta (Azure OpenAI) + composant suta-chatbot.tsx.
Score de confiance locataire	Implémentée (partielle)	Route /api/scoring ; Edge Function tenant-scoring héritée (lit une table profiles obsolète — cf. section 9.1).

2.5 Flux utilisateurs clés

2.5.1 Parcours du locataire

Le parcours de bout en bout est le suivant, illustré par la figure 2 :

1. Inscription : POST /api/auth/register (rôle whitelisté côté serveur depuis le durcissement SEC-001), puis vérification de l'e-mail ou du téléphone par OTP (ANSUT).
2. KYC : vérification d'identité par NNI auprès du RNPP Connect (ex-ONECI) et authentification faciale NeoFace ; les colonnes users.oneci_verified / neoface_verified et les tables d'historisation sont mises à jour.
3. Recherche : catalogue public GET /api/properties, favoris et alertes de recherche.
4. Dossier de location : création via POST /api/rental-file (types STANDARD, FAMILLE ou COLOCATION — un seul brouillon par type), ajout de documents, puis soumission.
5. Validation par le tiers de confiance : /api/tc/rental-files, avec suivi des engagements de délai (validation_slas).
6. Candidature : POST /api/applications avec le dossier validé ; unicité d'une candidature active par bien.
7. Bail : création par le bailleur (POST /api/leases/create, dossier validé requis), puis signature électronique : request-sign-otp (OTP CRYPTONEO par e-mail ou SMS), sign (signFileBatch + vérification), génération du PDF signé et activation du bail.
8. Paiements : loyers générés mensuellement (Edge Function planifiée generate-monthly-payments, rappels J-3 / J-1), règlement par Orange Money, MTN MoMo, Moov Money ou Wave (initiate + webhook HMAC), gestion des paiements partiels.
9. Quittance : GET /api/payments/[id]/quittance — quittance numérotée, immuable, accessible au locataire, au propriétaire et à l'administrateur.
10. Fin de bail : résiliation, états des lieux de sortie (agent de terrain), restitution du dépôt (deposit_refunds), avis croisés modérés.



Figure 2 — Parcours locataire de bout en bout (7 étapes clés)

2.5.2 Parcours du bailleur (propriétaire)

Le propriétaire publie un bien (POST /api/properties, statut DRAFT puis PENDING_VERIFICATION), dépose son dossier propriétaire (owner-file) et les titres de propriété (ownership_documents) qui sont validés par le tiers de confiance — éventuellement après une mission de vérification physique. Une fois le bien ACTIF (expiration automatique des annonces à 90 jours), il reçoit les candidatures et demandes de visite, peut contre-proposer un créneau, sélectionner un locataire, créer le bail et le signer par le même flux CRYPTONEO. Il suit ensuite les paiements et ses finances (y compris les paiements partiels), décide de la restitution du dépôt de garantie avec justificatif, répond aux avis et peut déléguer la gestion à une agence par mandat (cf. section 2.5.3).

2.5.3 Parcours de l'agence

L'agence opère un portefeuille de biens sous mandat. Le mandat de gestion (table mandats : GESTION_COMPLETE, GESTION_LOCATION ou MANDAT_SIMPLE) est signé électroniquement par le propriétaire et l'agence (OTP CRYPTONEO, colonnes owner/agency_signature_image et cryptoneo_operation_id). Une fois le mandat ACTIF, l'agence gère candidatures, visites et contrats pour le compte du propriétaire ; à l'activation de chaque bail de son portefeuille, une commission est créée automatiquement (table commissions, taux en pourcentage ou montant fixe). L'espace agence inclut également la gestion de l'équipe interne (agency_agents avec rôles ADMIN / AGENT / READ_ONLY et agents connectables depuis la migration 20261027000030) et les finances.

2.5.4 Parcours de l'agent de terrain

L'agent de terrain est un intervenant sans compte, piloté par un tiers de confiance. Le flux (figure 3) est le suivant : le TC crée la fiche agent (verification_agents) et la mission (PROPERTY_VERIFICATION ou INVENTORY_REPORT) ; il notifie l'agent par SMS ou e-mail (ANSUT) et génère un lien magique (POST /api/tc/missions/[id]/access-link) — jeton de 32 octets haché en sha256 avec le pepper MISSION_TOKEN_PEPPER, à durée de vie de 1 à 168 heures, affiché en clair une seule fois. L'agent ouvre la page publique /m/[token] (hors middleware, noindex), consulte la fiche bornée à sa mission, démarre la mission (statut IN_PROGRESS), remplit la checklist, joint ses photos (bucket privé mission-evidence en URLs signées) et soumet (COMPLETED). Le TC suit les badges envoyé / ouvert / rendu, peut régénérer le lien (révocation des anciens) et valide le rapport. Chaque étape est journalisée (audit_logs : MISSION_TOKEN_SENT / OPENED / SUBMITTED).



Figure 3 — Flux agent de terrain à jeton éphémère (TC-001, Option C)

2.5.5 Parcours de l'administrateur

L'administrateur supervise la plateforme : tableau de bord consolidé, gestion des utilisateurs et des comptes TC, modération des biens et des avis, traitement des signalements et des litiges, sécurité (journaux d'audit et de connexions, sessions), sauvegardes (table backups avec restauration et téléchargement), et configuration (platform_settings, drapeaux de fonctionnalités gérés par Edge Functions manage-feature-flags / check-feature-flag, règles métier manage-business-rules / get-business-rule).

3. Architecture technique

3.1 Vue d'ensemble en couches

La plateforme suit une architecture en cinq couches (figure 1) : les clients (navigateur et application mobile Capacitor) consomment une application Next.js 16 unique, qui s'appuie sur Supabase pour les données, l'authentification, le stockage et le temps réel ; les traitements serveur sensibles sont isolés dans des Edge Functions Deno, qui appellent les partenaires externes. Le runtime applicatif est un serveur Next.js standalone supervisé par PM2 derrière un reverse proxy Caddy.

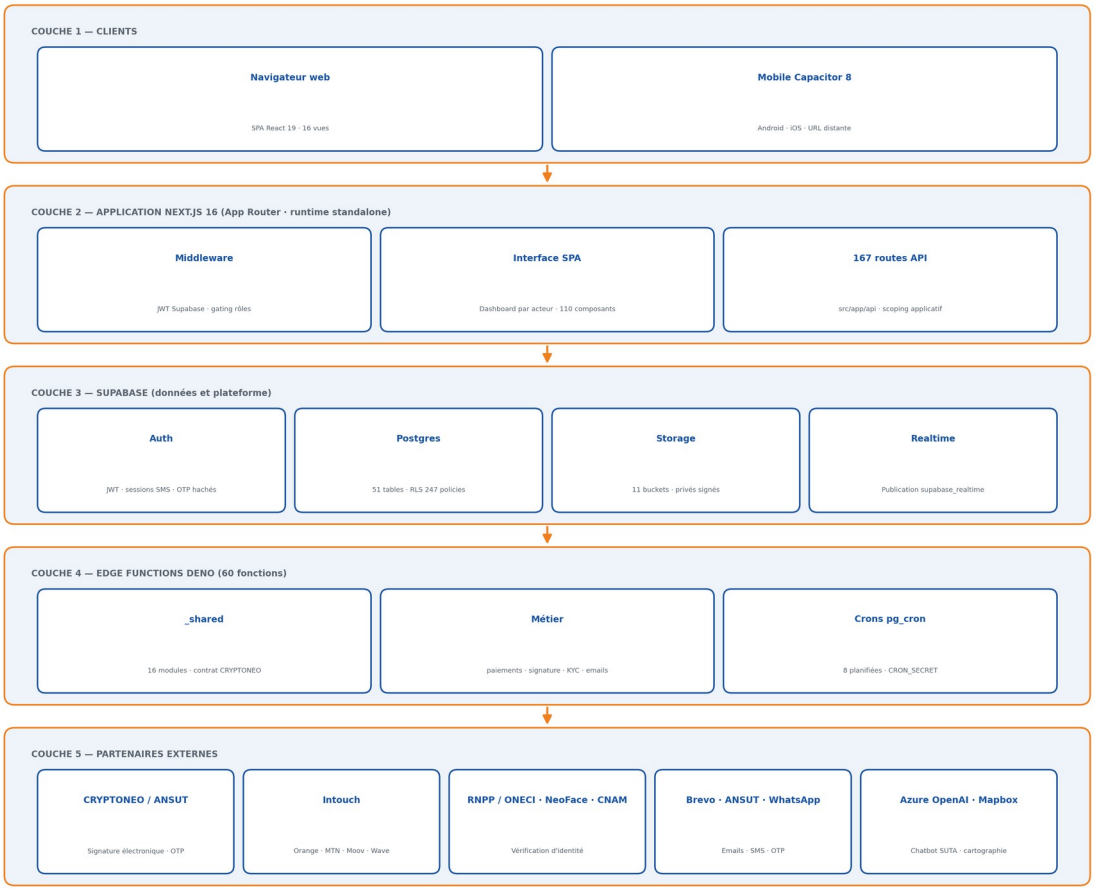


Figure 1 — Architecture technique en couches de la plateforme

3.2 Stack technique

Tableau 5 — Stack technique (extrait de package.json)

Domaine	Technologie	Version
Framework	Next.js (App Router, output standalone)	^16.2.6
Interface	React + React DOM	^19.0.0
Styles	Tailwind CSS v4 (PostCSS) + shadcn/ui (style New York)	^4
Données client	@supabase/supabase-js + @supabase/ssr	^2.106.0 / ^0.10.3
État serveur / client	TanStack React Query / Zustand	^5.82.0 / ^5.0.6
Validation	Zod + react-hook-form (+ resolvers)	^4.0.2 / ^7.60
Cartographie	Leaflet + react-leaflet (+ Mapbox côté serveur)	1.9 / ^5.0

Domaine	Technologie	Version
Graphiques	Recharts	^2.15.4
Animations	Framer Motion	^12
Documents	docx (baux et documents)	^9.6.1
Mobile	@capacitor/core / android / ios	^8.3.4
Qualité	TypeScript, ESLint, Prettier, Husky, lint-staged	TS ^5, ESLint ^9
Tests	Vitest (+ config sécurité dédiée), Playwright	^4.1.8 / ^1.60
Infrastructure locale	PM2 (ecosystem.config.js) + Caddy	—

3.3 Frontend — application mono-page

L'interface applicative est une SPA : la page unique src/app/page.tsx (composant client) dispatche seize vues selon l'état currentView du store d'authentification (home, nos-biens, a-propos, faq, nous-contacter, cgu, confidentialite, mentions-legales, cookies, login, register, otp-verify, email-verify, forgot-password, dashboard, property-detail) et gère l'historique manuellement (popstate). Le dashboard est un routeur client par rôle actif (src/components/dashboard/index.tsx) : environ 110 composants répartis dans les cinq dossiers acteurs. La page publique /m/[token] est la seule page dédiée hors SPA (rapport de mission d'agent de terrain).

L'état et les accès aux données s'organisent en couches : contexts (AuthContext, RoleContext, ThemeContext) et store Zustand (auth-store, active-dossier-store) ; TanStack Query pour l'état serveur (migration progressive FE-003, ADR-0003) ; 33 hooks temps réel use-realtime-* (applications, baux, paiements, mandats, missions, messages, notifications, etc.) ; services métier par domaine sous src/services/ (contracts, mandates, payments, brevo, verification, cryptonoe, tc, azure, ai, storage, notifications, scoring…). L'UI s'appuie sur shadcn/ui (style New York) et Tailwind v4, avec thème clair / sombre (next-themes).

3.4 Couche API Next.js

167 fichiers route.ts composent l'API sous src/app/api. Les handlers suivent un pattern commun : résolution de l'utilisateur (resolveRequestUser — JWT vérifié ou session SMS), scoping des requêtes par rôle et par propriété, journalisation audit_logs des opérations sensibles. Le tableau ci-dessous regroupe les domaines.

Tableau 6 — Domaines de l'API (src/app/api)

Domaine	Exemples de routes
Authentification	auth/login, auth/register, auth/logout, auth/me, auth/forgot|reset-password, auth/send-email|sms-otp, auth/verify-email|sms|phone-otp
Biens et catalogue	properties (+[id], documents, inventory-reports, signal, reviews), favorites, search-alerts, stats
Dossiers et candidatures	rental-file (+documents, withdraw), rental-files/[id]/action, applications (+[id]), certificates
Baux et paiements	leases (+create, contract, sign, request-sign-otp, cancel, terminate, deposit-refund), payments (+initiate, callback, balance, advance, paiement-immediat, check-overdue, [id]/quittance)
Signature électronique	signature/auth, signature/send-otp, signature/sign, signature/verify, signature/signed-file, signature/generate-certificate, signature/sync-aliases
Mandats	mandats (+[id], [id]/sign, [id]/request-sign-otp, [id]/pdf)
Tiers de confiance	tc/agents, tc/certifications, tc/fraud-alerts, tc/litiges, tc/missions (+access-link, evidence), tc/oneci, tc/ownership-docs, tc/properties, tc/rental-files, tc/users, tc/verifications
Agents de terrain (public)	missions-token/[token] (accès à jeton, page /m/[token])
KYC et identité	kyc/face-auth, kyc/oneci/{person, match, face-auth, fingerprint-auth, subscription}, oneci/verify
Messagerie et notifications	messages (+send, contacts), notifications, maintenance (+comments)
Espaces acteurs	dashboard/{locataire, proprietaire, agence, tc, admin}, owner/{analytics, reviews, rental-files, finances}, owner-file, tenants, locataire/my-recipients

Domaine	Exemples de routes
Administration et plateforme	admin/{users, backups, signalements, properties-moderation, reviews, settings, system, reports, audit-logs}, agence/{users, agents, commissions, legal-documents, settings}, user/{2fa, change-password, connection-logs, delete-account, switch-role, …}, settings, profile, renewals, reviews, scoring, suta, users/search, visits, disputes, history

3.5 Middleware et contrôle d'accès

Le middleware racine crée un client Supabase SSR (@supabase/ssr) pour chaque requête, vérifie le jeton par supabase.auth.getUser() et résout le rôle actif (getUserRole). Il applique ensuite la cartographie ROLE_ROUTES : chaque espace d'API d'acteur n'est servi qu'au rôle actif correspondant. Les préfixes publics (auth, listing des biens, avis) et les préfixes protégés (session requise) complètent la cartographie. Une session SMS alternative est acceptée via le cookie httpOnly montoit-session (token aléatoire de 48 octets, 30 jours glissants, SameSite=Lax), rattaché à la table sessions.

3.6 Modèle de données

Le socle de données est un Postgres Supabase décrit par 102 migrations. Les 36 enums couvrent les rôles, statuts et types métier (propriétés, dossiers, baux, paiements, missions, mandats, litiges…). Le temps réel est assuré par la publication supabase_realtime (complétée par la migration 20261027000002) sur les tables métier. Les tables clés et leurs relations sont les suivantes :

Tableau 7 — Tables principales du modèle de données

Table	Colonnes et relations principales
users	id (= auth.users), phone, email, role + active_role, is_active, vérifications email / téléphone, KYC : nni, oneci_verified(+_at), neoface_verified(+_at), kyc_document_id ; colonnes sensibles protégées par trigger (SEC-011).
properties	title, type (APPARTEMENT … VILLA), status (DRAFT → PENDING_VERIFICATION → ACTIVE → SUSPENDED / CLOSED / RENTED), rental_status, price, area, bedrooms, adresse (city, commune, latitude / longitude), équipements, owner_id → users.

Table	Colonnes et relations principales
rental_files	status (DRAFT → SUBMITTED → TC_REVIEW → VALIDATED → ACCEPTED / REJECTED / EXPIRED), tenant_category, monthly_income, garant, file_type (STANDARD / FAMILLE / COLOCATION — un brouillon par type), tenant_id, reviewed_by_id.
applications	rental_file_id → rental_files, property_id → properties, tenant_id ; unicité d'une candidature active par bien.
leases	status (DRAFT → PENDING_SIGNATURE → ACTIVE → TERMINATED / EXPIRED / CANCELLED), dates, monthly_rent, charges, deposit, signatures propriétaire / locataire (otp + image + horodatage), property_id, tenant_id, owner_id, rental_file_id, contract_url, cryptoneo_operation_id.
payments	amount + amount_paid (PARTIAL), status (PENDING / PAID / LATE / PARTIAL / CANCELLED / PROCESSING), due_date, reference Intouch, method (ORANGE_MONEY, MTN_MOMO, MOOV_MONEY, WAVE), lease_id, tenant_id ; table sœur payment_attempts (idempotence).
rent_receipts	payment_id (unique), lease_id, receipt_number (QT-YYYY-NNNNNN, séquence SECURITY DEFINER réservée service_role), période, montant, instantanés jsonb (locataire, bailleur, bien) ; immuable, INSERT service_role uniquement.
mandats	type (GESTION_COMPLETE, GESTION_LOCATION, MANDAT_SIMPLE), status, commission (taux ou montant fixe), property_id, owner_id, agency_id, contract_url, images de signature, cryptoneo_operation_id.
missions / verification_agents	mission : type (PROPERTY_VERIFICATION, INVENTORY_REPORT), status (ASSIGNED → IN_PROGRESS → COMPLETED), property_id, agent_id → verification_agents, tc_id, inventory_report_id (unique). Fiche agent : identité, téléphone, is_active, tc_id — sans compte utilisateur.

Table	Colonnes et relations principales
mission_access_tokens	mission_id (cascade), token_hash (sha256 + pepper, unique — le token clair n'est jamais stocké), channel (SMS / EMAIL), expires_at, opened_at, submitted_at, revoked_at, created_by ; écritures réservées service_role.
agency_agents / commissions	rôles internes (ADMIN, AGENT, READ_ONLY), user_id connectable (migration 20261027000030), affectations de biens sécurisées ; commissions : montant, taux, statut, mandat_id, lease_id, agency_id.
deposit_refunds	lease_id (unique), deposit_amount, refund_amount, deductions (+ motif), justification_url, status (PENDING / DECIDED / PAID), decided_by.
signature_aliases	user_id (unique), alias_certificat (unique), identité, email, téléphone, organisation, données de certificat, is_active — miroir local des certificats CRYPTONEO.
KYC et historisation	oneci_verifications (RNPP), facial_verifications (NeoFace) : résultats, statut et horodatage des vérifications d'identité.
Transverses	notifications (+ notification_preferences), conversations / messages / message_attachments, favorites, visit_requests, ratings, disputes, signalements, fraud_alerts, audit_logs, connection_logs, sessions, otp_codes, validation_slas (polymorphe), inventory_reports (+ items), maintenance_requests (+ comments), owner_files (+ documents), ownership_documents, property_documents / images, certifications, service_usage_logs, platform_settings, default_conditions, backups, recurring_payments, email_templates.

3.7 Authentification et sessions

La plateforme combine deux systèmes d'authentification : (1) le JWT Supabase Auth, vérifié cryptographiquement dans le middleware et les routes (resolveRequestUser) ; (2) une session SMS personnalisée — cookie httpOnly montoit-session (token de 48 octets, 30 jours glissants) rattaché à la table sessions. Les codes OTP (connexion, vérification e-mail / téléphone, signature) sont stockés hachés dans otp_codes avec le pepper OTP_HASH_PEPPER (src/lib/otp-hash.ts). Les jetons de mission des agents de terrain

obéissent au même principe avec le pepper MISSION_TOKEN_PEPPER : absence de pepper ⇒ échec fermé (503). Une double authentification (2FA) et la gestion des sessions actives sont exposées côté utilisateur (/api/user/2fa, settings/sessions, connection-logs).

3.8 Edge Functions Supabase

Soixante Edge Functions Deno implémentent les traitements serveur sensibles, appuyées sur seize modules partagés (_shared) : contrat CRYPTONEO (cryptoneo.ts, cryptoneo-contract.ts), Intouch (intouch.ts), RNPP (oneci.ts), messagerie ANSUT (ansut-messaging.ts), auth (auth.ts), clients Supabase admin, CORS, HMAC, journalisation, détection Cloudflare, types partagés. Huit fonctions sont planifiées par pg_cron (section 6.5).

Tableau 8 — Edge Functions par domaine

Domaine	Fonctions (noms exacts)
Paiements Intouch (8)	payment-initiate, payment-callback (webhook HMAC fail-closed), payment-transfer (paiement immédiat), initiate-payment, generate-monthly-payments, generate-receipt, check-overdue-payments, remind-upcoming-payments.
Signature CRYPTONEO courante (8)	signature-auth, sign-send-otp, sign, sign-verify, signed-file, generate-certificate, retrieve-certificates, check-certificate.
Signature CRYPTONEO legacy (6)	cryptoneo-auth, cryptoneo-send-otp, cryptoneo-sign-document, cryptoneo-verify-signature, cryptoneo-sign-mandate, cryptoneo-generate-certificate — génération historique référençant des tables absentes des migrations (cf. section 9.1).
KYC identité (9)	oneci-person, oneci-match, oneci-face-auth, oneci-fingerprint-auth, oneci-subscription, oneci-verification (RNPP Connect) ; kyc-face-auth, neoface-verify (NeoFace) ; cnam-verification.
Communications (14)	send-email, send-email-brevo, send-sms-azure, send-verification-otp, verify-otp, verify-otp-azure, verify-email-otp, verify-reset-token, send-password-reset, send-whatsapp-otp, send-whatsapp-hybrid, send-lease-notifications, send-mandate-notifications, send-application-notifications.

Domaine	Fonctions (noms exacts)
Plateforme et divers	tenant-scoring, ai-chatbot, check-feature-flag, manage-feature-flags, get-business-rule, manage-business-rules, get-ip, get-mapbox-token, get-verification-image, process-agent-registration, remind-upcoming-visits, expire-stale-listings, expire-stale-rental-files, remind-lease-expiring, remind-pending-lease-signatures.

3.9 Stockage (Supabase Storage)

Onze buckets structurent les fichiers. Les buckets documentaires sont privés depuis le durcissement SEC-006 (migration 20261027000001) : l'accès passe par URLs signées d'une heure, avec policies par dossier utilisateur (chemin <userId>/...) et accès administrateur / TC / agence selon le cas.

Tableau 9 — Buckets Storage

Bucket	Visibilité	Contenu
avatars · property-images · property-videos	Public	Catalogue et profils (images / vidéos de biens, avatars).
owner-documents · rental-documents · property-documents · lease-documents · mandat-documents · maintenance-images	Privé	Pièces d'identité et dossiers, documents de dossiers de location, titres de propriété, contrats signés, images de maintenance.
mission-evidence	Privé	Preuves photographiques des missions d'agents de terrain (TC-001, migration 20261027000060).
message-attachments	Privé	Pièces jointes de la messagerie (migration 20261027000040).

3.10 Sécurité

Le modèle de sécurité combine quatre niveaux :

1. Contrôle d'accès applicatif : middleware de rôles, résolution d'utilisateur vérifiée, scoping systématique dans les 167 routes API. Conformément à l'ADR-0002 (assumé), 147 routes utilisent le client service_role — la RLS est donc contournée par design côté API et la sécurité applicative constitue la première ligne de défense ; la migration vers le client SSR est planifiée en trois phases (Phase 1 : users, otp_codes,

sessions et colonnes KYC — service_role légitime ; Phase 2 : payments / leases en client SSR).

2. RLS en profondeur : 247+ politiques activées sur toutes les tables métier (durcissements SEC-004 sur users, SEC-011 — trigger anti-édition de neuf colonnes KYC par leur titulaire, policies dédiées applications, property_documents, agency_agent_properties…).
3. Secrets et jetons : OTP hachés avec pepper, tokens de mission hachés (jamais stockés en clair), webhook de paiement fail-closed vérifié par HMAC timing-safe (x-webhook-signature, PAYMENT_CALLBACK_SECRET), vérification JWT activée sur les Edge Functions métier (SEC-008, runbook de classification).
4. Durcissement HTTP : en-têtes CSP, HSTS, X-Frame-Options DENY, Permissions-Policy (caméra limitée à self) posés par next.config.ts ; buckets documentaires privés avec URLs signées d'une heure.

Les runbooks docs/RUNBOOK-EDGE-FUNCTIONS-SECURITE.md, docs/RUNBOOK-ROTATION-SECRETS.md et docs/RUNBOOK-CRONS.md couvrent l'exploitation sécurité (classification verify_jwt, rotation des secrets, planification).

4. Intégrations externes

4.1 CRYPTONEO / ANSUT — signature électronique

La signature électronique des baux et des mandats est déléguée à la plateforme CRYPTONEO / ANSUT. Le contrat partagé _shared/cryptoneo-contract.ts (source de vérité DAT-004, importée par le runtime Node via src/lib/cryptoneo.ts et par Deno via _shared/cryptoneo.ts) fige les formes d'API, les codes de succès ({200, 7000, 7002, 7003, 7004}) et la durée de vie du jeton (30 minutes, marge 60 secondes). Le flux baux / mandats : résolution ou création de l'alias de certificat (table signature_aliases, génération via /generateCert/generateCertificat avec consentement), envoi de l'OTP (/otp/send, canal e-mail ou SMS), signature par lots (/sign/signFileBatch avec alias, OTP et requêtes de signature), vérification de l'opération (/sign/verifySignedBatch — succès 7004), récupération du PDF signé, archivage dans le bucket lease-documents ou mandat-documents, puis activation (bail → notifications + commission éventuelle d'agence).

Tableau 10 — Endpoints CRYPTONEO consommés

Endpoint	Méthode	Usage
/user/auth	POST	Authentification de l'application (appKey + appSecret) — jeton JWT en cache 30 minutes.

Endpoint	Méthode	Usage
/generateCert/generateCertificat	POST	Génération du certificat numérique de l'utilisateur (identité, photo, consentement daté, organisation MONTOIT).
/generateCert/users	GET	Recherche d'alias de certificats — synchronisation de la table signature_aliases (admin).
/otp/send	POST	Envoi du code de signature par e-mail ou SMS (typeOperation SIGNATURE_ELECTRONIQUE).
/sign/signFileBatch	POST	Signature par lots (aliasCertificat, otp, signRequest[] : urlDoc, hashDoc, image de signature, lieu Abidjan) + callBackUrl.
/sign/verifySignedBatch	POST	Vérification du statut de l'opération (operationId) — succès 7004.
/sign/getSignedFile	GET	Récupération du document signé.

Les variables dédiées sont CRYPTONEO_API_URL (défaut https://signature.ansut.ci ; la valeur Edge par défaut historique est https://ansut.cryptoneoplatforms.com/esignaturedemo), CRYPTONEO_APP_KEY et CRYPTONEO_APP_SECRET. La documentation d'implémentation détaillée de cette intégration fait l'objet d'un document séparé : docs/cryptoneo/Implementation_CRYPTONEO_MonToitCI.docx.

4.2 Intouch — paiements mobile money

Les encaissements et transferts passent par la passerelle Intouch (apidist.gutouch.net pour les paiements, businessapi.gutouch.com pour les paiements immédiats). Le module partagé _shared/intouch.ts expose initiatePaiement, checkTransactionStatus, initiatePaiementImmediat, getBalance et la génération d'identifiants de transaction (références LOYER-, CAUTION-, AVANCE-, AGENCE-). Quatre opérateurs sont supportés : Orange Money, MTN MoMo, Moov Money et Wave (redirection par URL pour ce dernier). Le webhook payment-callback est sécurisé par HMAC (fail-closed) ; les transferts vers les bénéficiaires (propriétaire, agence) passent par payment-transfer avec plafond optionnel MAX_TRANSFER_AMOUNT_XOF.

4.3 Vérification d'identité (KYC)

Trois fournisseurs composent la chaîne KYC : RNPP Connect (ex-ONECI) pour la vérification de la pièce d'identité par NNI (person, match, face-auth, fingerprint-auth, subscription — _shared/oneci.ts, historisation oneci_verifications) ; NeoFace (AIneo) pour l'authentification faciale (kyc-face-auth, neoface-verify, historisation facial_verifications) ; et une Edge Function cnam-verification (couverture CNAM, à confirmer en exploitation — cf. section 9.1). Les crédits RNPP sont surveillés (balance) et la rotation de la clé fait l'objet d'une action d'exploitation (runbook).

4.4 Communications (e-mail, SMS, WhatsApp)

Tableau 11 — Fournisseurs de communications

Canal	Fournisseur	Usages
E-mail transactionnel	Brevo (API v3)	Templates, OTP e-mail, notifications (Edge Function send-email-brevo ; services src/services/brevo/*).
SMS	ANSUT Hub	OTP de connexion et de vérification, notifications de missions aux agents de terrain (notifyAgentOfMission), expéditeur ANSUT.
SMS (secondaire)	Azure Communication Services	Edge Function send-sms-azure ; OTP de signature historique.
WhatsApp	Passerelle WhatsApp	send-whatsapp-otp, send-whatsapp-hybrid (OTP et notifications hybrides).
Notifications applicatives	Supabase Realtime	Tables notifications + notification_preferences, hooks use-realtime-notifications.

4.5 Intelligence artificielle

Deux services Azure complètent la plateforme : Azure OpenAI (déploiement gpt-5.6-luna) alimente l'assistant conversationnel SUTA (route /api/suta, composant suta-chatbot.tsx), et Azure Translator (src/services/azure/) assure la traduction de contenus.

4.6 Cartographie et géolocalisation

Les cartes des biens utilisent Leaflet côté client ; le jeton Mapbox est servi par Edge Function (get-mapbox-token) pour les composants MapboxMap. Les coordonnées des biens

(properties.latitude / longitude) alimentent la recherche géographique. Côté mobile, le plugin Capacitor Geolocation (hooks useNativeGeolocation, use-geolocation) permet la géolocalisation native.

5. Application mobile — Capacitor 8

5.1 Configuration

Tableau 12 — Configuration Capacitor (capacitor.config.ts)

Paramètre	Valeur
appId / appName	com.montoit.app / Mon Toit
webDir	out (voir ADR-0004 : l'export statique n'est pas produit ; l'application charge l'URL distante)
server.url	https://mon-toit.ansut.ci (WebView pointant vers le site déployé)
androidScheme / cleartext	https / true (allowMixedContent)
allowNavigation	mon-toit.ansut.ci, neoface.aineo.ai, *.aineo.ai
errorPath	offline.html (page hors-ligne)
StatusBar	overlaysWebView false, style DARK, fond #FFFFFF
LocalNotifications / Push	Icône ic_stat_icon_config, couleur #F57C00, badge / son / alerte

5.2 Plugins et habillage natif

Le shell natif intègre les plugins StatusBar, LocalNotifications, PushNotifications, App (gestion du bouton retour), Browser / InAppBrowser, Geolocation, Network et AppLauncher. Le projet Android (Gradle complet) et le projet iOS (Xcode, splash natif personnalisé dans MainViewController.swift) sont présents dans le dépôt. Côté JavaScript, neuf hooks capacitor/* et sept hooks natifs complètent l'habillage : StatusBarController, NativeCameraUpload, SafeAreaWrapper, app-back-handler, app-lifecycle-manager, offline-banner, usePushNotifications.

5.3 Stratégie de build (ADR-0004)

Deux options sont documentées : l'option A (retenue par défaut) charge l'URL distante dans la WebView — le webDir out n'est jamais produit ; l'option B (export statique via BUILD_TARGET=capacitor) est conditionnée à une publication sur les stores publics, la variable isCapacitorIsBuild étant calculée mais sans effet dans next.config.ts à ce jour. La

décision produit reste ouverte (ADR-0004 au statut PROPOSÉ). Les scripts cap:sync / cap:copy / cap:build / cap:build:apk automatisent la chaîne Android.

6. Qualité, CI/CD et exploitation

6.1 Chaîne d'intégration continue

Le workflow .github/workflows/ci.yml (déclencheurs : push et pull request sur 4.0.0 et main) exécute six jobs avec Bun 1.3.14 :

Tableau 13 — Jobs de la CI GitHub Actions

Job	Contenu
install	Installation des dépendances en mode --frozen-lockfile.
lint	Analyse ESLint (règles Next.js et qualité).
typecheck	Contrôle TypeScript strict (tsc --noEmit).
test	Suite Vitest (tests unitaires et d'intégration).
secrets-scan	Détection de secrets dans le dépôt (grep inline + liste d'exclusion).
audit	Audit des dépendances (continue-on-error).

6.2 Conventions de développement

- Husky : pre-commit → lint-staged ; commit-msg → Conventional Commits (types feat, fix, docs, style, refactor, test, chore, perf, ci, build, revert ; objet ≤ 72 caractères, guide COMMIT_GUIDELINES.md).
- Prettier et ESLint 9 (flat config) sur l'ensemble du dépôt ; TypeScript strict.
- Migrations SQL immuables (jamais éditées après application) ; règle d'idempotence documentée dans .ai/LESSONS_LEARNED.md (pas de pattern check-then-insert non protégé).

6.3 Stratégie de tests

- Vitest (environnement jsdom, setup src/test/setup.ts) : tests unitaires et d'intégration — 44 tests verts à la fin de la wave 3 (dont 44 pour le helper des quittances et les constantes de rôles).
- Configuration de sécurité dédiée vitest.security.config.ts + tests security.test.ts et penetration-test.cjs.

- Playwright (Chromium, workers 1, serveur de développement sur le port 3000) : parcours e2e/auth-flow.spec.ts et e2e/roles.spec.ts ; scénarios E2E complémentaires planifiés (TC-007, FUN-007, FUN-012 — bloqués par l'accès Supabase).

6.4 Déploiement et exploitation

Le build de production produit un serveur Next.js standalone (next build + copy-standalone-assets.mjs), exécuté sous PM2 (application montoit, port 3000, ecosystem.config.js) derrière le reverse proxy Caddy (entrée :81 → 3000). Aucun conteneur n'est défini (pas de Dockerfile) ; le fichier vercel.json est un vestige Vite inadapté (cf. section 9.1). Les rappels métier passent par les Edge Functions planifiées ci-dessous.

6.5 Tâches planifiées (pg_cron)

Tableau 14 — Edge Functions planifiées

Fonction	Programmation	Rôle
generate-monthly-payments	0 0 1 * *	Génération des échéances de loyer du mois.
check-overdue-payments	30 8 * * *	Détection des impayés (statut LATE).
remind-upcoming-payments	0 8 * * *	Rappels d'échéance (J-3 / J-1).
remind-upcoming-visits	0 8 * * *	Rappels de visites à venir.
remind-lease-expiring	0 8 * * *	Rappels de baux arrivant à échéance (renouvellements).
remind-pending-lease-signatures	0 8 * * *	Relances des signatures de baux en attente (48 h, idempotent reminder_sent_at).
expire-stale-listings	30 0 * * *	Expiration automatique des annonces à 90 jours.
expire-stale-rental-files	30 0 * * *	Expiration des dossiers de location périmés.

6.6 Pilotage projet (.ai/)

Le dossier .ai/ concentre le pilotage : ACTORS.md (acteurs et matrice de permissions), ARCHITECTURE.md (choix structurants A1-A7, registre de dette S1-S9 / T1-T10), ADRs/ (décisions d'architecture), SPECS/ (FUN-001, FUN-002, TC-001), ANALYSES/ (audits 1-a à 1-e : structure, fonctionnel, sécurité, frontend, données), TEAM_STATUS.md

(avancement par waves, backlog 48/54 tâches validées à la fin de la wave 3) et LESSONS_LEARNED.md.

7. Environnement et configuration

La configuration est décrite par .env.local.example, organisé en neuf sections. Les secrets sont gérés hors dépôt (variables locales pour Next.js, secrets Supabase pour les Edge Functions — procédure de rotation : docs/RUNBOOK-ROTATION-SECRETS.md).

Tableau 15 — Variables d'environnement par section

Section	Variables principales
Supabase	NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY, SUPABASE_SERVICE_ROLE_KEY
Sécurité applicative	OTP_HASH_PEPPER, MISSION_TOKEN_PEPPER, PAYMENT_CALLBACK_SECRET, CRON_SECRET
Application	NEXT_PUBLIC_APP_URL, NEXT_PUBLIC_CAPACITOR_API_URL, OTP_EXPIRY_MINUTES
Intouch (paiements)	INTOUCH_BASE_URL, INTOUCH_USERNAME, INTOUCH_PASSWORD, INTOUCH_PARTNER_ID, INTOUCH_LOGIN_API, INTOUCH_API_PASSWORD, INTOUCH_CALLBACK_URL, INTOUCH_CALLBACK_SECRET, INTOUCH_PAIEMENT_IMMEDIAT_URL, INTOUCH_WAVE_RETURN_URL, INTOUCH_WAVE_CANCEL_URL, PAYMENT_CALLBACK_SECRET, MAX_TRANSFER_AMOUNT_XOF
CRYPTONEO (signature)	CRYPTONEO_API_URL, CRYPTONEO_APP_KEY, CRYPTONEO_APP_SECRET (+ CRYPTONEO_BASE_URL historique lu par les fonctions legacy)
RNPP Connect (ex-ONECI)	RNPP_API_BASE, RNPP_API_KEY

Section	Variables principales
NeoFace (facial)	NEOFACE_API_BASE, NEOFACE_BEARER_TOKEN
ANSUT (SMS / e-mail)	ANSUT_API_BASE_URL, ANSUT_API_USERNAME, ANSUT_API_PASSWORD, ANSUT_SMS_SENDER
Azure et Brevo	AZURE_OPENAI_ENDPOINT, AZURE_OPENAI_API_KEY, AZURE_OPENAI_DEPLOYMENT_NAME, AZURE_OPENAI_API_VERSION ; AZURE_SMS_URL / USERNAME / PASSWORD / FROM ; BREVO_API_KEY, BREVO_SENDER_EMAIL, BREVO_SENDER_NAME

8. Décisions d'architecture (ADRs)

Quatre décisions structurantes sont formalisées dans .ai/ADRs/ :

Tableau 16 — Registre des décisions d'architecture

ADR	Sujet	Décision	Statut
ADR-0001	Routage applicatif	Conserver la SPA mono-page ; migration progressive conditionnée (lot public ~2 jours si SEO, dashboard par lots de 10 écrans si critères T1-T3).	DÉCIDÉ

ADR	Sujet	Décision	Statut
ADR-0002	Autorisation	Modèle hybride en trois phases : service_role assumé côté API avec sécurité applicative (Phase 1 : users / otp_codes / sessions / PII KYC — service_role légitime), RLS en défense en profondeur, puis migration payments / leases vers le client SSR (Phase 2).	DÉCIDÉ
ADR-0003	Couche de données	React Query + services par acteur ; objectif zéro appel authFetch direct dans les composants (règles R1-R5, migration FE-003).	DÉCIDÉ
ADR-0004	Mobile Capacitor	Option A retenue par défaut : WebView sur URL distante ; Option B (export statique) réservée aux stores publics — décision produit requise.	PROPOSÉ

9. Dette technique, points d'attention et évolutions

9.1 Points d'attention identifiés

Tableau 17 — Points d'attention de l'architecture actuelle

Point	Détail	Action recommandée
Arbres applicatifs dupliqués (environnement de travail)	Le working tree local contient, en plus de l'arbre versionné src/app/, un dossier racine app/ et un src/pages/ non versionnés (artefacts de l'environnement d'exécution). Next.js 16 refuse de démarrer si les deux arbres coexistent (pages et app sous des dossiers différents).	Documenter src/app/ comme arbre de vérité ; ne jamais versionner les artefacts racine app/ et src/pages/.
Edge Functions CRYPTONEO legacy	Les six fonctions cryptoneo-* référencent des tables absentes des migrations (lease_contracts, profiles, digital_certificates, admin_audit_logs) ; le flux réel passe par signature-auth / sign-send-otp / sign / sign-verify.	Purger ou archiver les fonctions legacy après vérification qu'aucun appel ne subsiste.
Edge Function tenant-scoring	Lit la table profiles, absente des migrations actuelles (utilisateurs portés par users).	Réécrire sur la table users ou remplacer par la route /api/scoring.
Edge Function cnam-verification	Présente mais non confirmée en exploitation (structures lues non vérifiées).	Vérifier le contrat réel CNAM avant mise en production.
vercel.json	Vestige Vite (rewrites vers /index.html), inadapté au runtime Next.js standalone.	Supprimer le fichier.
Couche contexts dupliquée	AuthContext / RoleContext coexistent avec le store Zustand auth-store et AuthProvider.	Converger vers une seule source de vérité d'authentification (ADR-0003).
Types Supabase	Trois générateurs / fichiers de types coexistent (types.ts manuel, database.types.ts, shared) — drift documenté (S9).	Régénérer les types depuis le schéma et unifier.

9.2 Restes à faire (TEAM_STATUS)

- TC-007, FUN-007, FUN-012 : scénarios E2E Playwright complémentaires (bloqués par l'accès à l'environnement Supabase).
- DOC-003 : revue juridique du dispositif de quittances.
- FE-003 (continu) : migration de la couche de données vers React Query + services par acteur (ADR-0003).
- Actions d'exploitation : rotation de la clé RNPP / ONECI, déploiement des migrations 20261027* sur l'environnement de production.

9.3 Axes d'évolution recommandés

- Phase 2 de l'ADR-0002 : bascule progressive des routes paiements / leases vers le client Supabase SSR pour ramener la RLS en première ligne de défense.
- Migrer les vues publiques vers le rendu serveur (App Router) pour le référencement naturel, conformément au lot 0 de l'ADR-0001, si le SEO devient prioritaire.
- Industrialiser le rappel asynchrone CRYPTONEO (callBackUrl) en complément du polling sign-verify, avec une Edge Function de callback dédiée.
- Étendre la CI avec les tests de parité du contrat CRYPTONEO (docs/DEDUP-LIB-EDGE-FUNCTIONS.md) à chaque évolution de _shared/cryptoneo-contract.ts.

10. Annexes

10.1 Documents de référence internes

- .ai/ACTORS.md — acteurs, matrice de permissions, risques transverses.
- .ai/ARCHITECTURE.md — vue d'ensemble, choix structurants, registre de dette.
- .ai/ADRs/0001-routing.md, 0002-authorization.md, 0003-data-layer.md, 0004-capacitor.md — décisions d'architecture.
- .ai/SPECS/ — FUN-001 (quittances), FUN-002 (dossiers multiples), TC-001 (agents terrain).
- .ai/TEAM_STATUS.md et .ai/LESSONS_LEARNED.md — avancement et leçons d'implémentation.
- docs/ENVIRONNEMENT.md — inventaire des variables d'environnement.
- docs/DATA-LAYER-GUIDE.md — guide de la couche de données client.
- docs/RUNBOOK-CRONS.md, docs/RUNBOOK-EDGE-FUNCTIONS-SECURITE.md, docs/RUNBOOK-ROTATION-SECRETS.md — runbooks d'exploitation.

- docs/DEDUP-LIB-EDGE-FUNCTIONS.md — contrat partagé CRYPTONEO et tests de parité (DAT-004).
- docs/AUDIT_MESSAGING.md — audit de la chaîne de messagerie.
- docs/cryptoneo/Implementation_CRYPTONEO_MonToitCI.docx — documentation dédiée de l'intégration de signature électronique.

10.2 Périmètre des migrations récentes (waves 2-3)

Tableau 18 — Migrations structurantes récentes

Migration	Contenu
20260910000001	Colonnes de signature des mandats (contract_url, images de signature, cryptoneo_operation_id).
20260910000003 / 000010 / 000011	Paiements : payment_attempts (idempotence), amount_paid et statut PARTIAL, table deposit_refunds.
20260911000002 / 000003 / 000004	KYC : oneci_verifications, correction RLS des logs KYC, RLS des documents de biens.
20261027000000 / 000050	SEC-004 : durcissement RLS de users ; SEC-011 : trigger anti-édition des colonnes KYC.
20261027000001 / 000040 / 000060	SEC-006 : buckets documentaires privés ; bucket message-attachments privé ; bucket mission-evidence privé.
20261027000002	DAT-003 : publication temps réel supabase_realtime complétée.
20261027000010	FUN-001 : table rent_receipts (quittances numérotées, immuables).
20261027000020	FUN-002 : enum rental_file_type + colonne file_type + index de brouillons.
20261027000030 / 000031 / 000032	TC-001 : user_id des agents d'agence + RLS ; table mission_access_tokens.

Fin du document.